USTU CAMPUS - INTEGRATED CAMPUS MANAGEMENT SYSTEM

COMPREHENSIVE MINI PROJECT REPORT

TITLE PAGE

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

Vidya Vikas Education Trust's

Universal Skilltech University, Vasai (E)

Department of Computer Science & Engineering

This is to certify that the mini project entitled **"USTU CAMPUS - Integrated Campus Management Platform"** is a bonafide work carried out by **Aarush, Amisa, Kritika, and Aayush** in partial fulfillment of the requirements for the Mini Project (Semester IV) of the Second Year Bachelor of Technology in Computer Science and Engineering program at Universal Skilltech University, Vasai, Mumbai, during the academic year 2025–2026 (Semester IV).

Supervisor & SE Coordinator

Pro-Vice Chancellor

MINI PROJECT APPROVAL

Examiners:*

(Internal Examiner Name & Sign)

(External Examiner Name & Sign)

TABLE OF CONTENTS

ABSTRACT

Modern educational institutions face significant challenges in managing diverse operational activities through fragmented systems. Departments operate in silos, utilizing separate platforms for attendance tracking, academic resource distribution, grade management, and institutional communication. This fragmentation creates inefficiencies, reduces accessibility, and hinders effective collaboration between students, faculty, and administrative personnel.

The USTU Campus platform presents a comprehensive, unified web-based solution designed to consolidate all campus operations into a single, integrated ecosystem. This full-stack application leverages contemporary web technologies including React with TypeScript for dynamic user interfaces, Supabase for robust database management, and Clerk for secure authentication protocols. The system accommodates multiple user roles—students, teachers, administrators, and canteen staff—each with customized dashboards and feature sets aligned with their specific operational requirements.

Key functionalities encompassing the platform include real-time attendance monitoring, comprehensive grade and performance tracking, dynamic resource and assignment management, integrated notice distribution, scheduling optimization, library resource coordination, and operational canteen management. The application implements role-based access control to ensure data security and appropriate permission hierarchies. With deployment on Vercel and powered by PostgreSQL databases, the system demonstrates scalability, reliability, and professional-grade performance suitable for institutional deployment.

The platform significantly improves operational efficiency by eliminating redundant processes, providing centralized data management, enabling seamless inter-departmental communication, and delivering an intuitive user experience across diverse user populations. This report presents the comprehensive system design, architectural decisions, implementation methodology, and technical specifications underpinning the USTU Campus platform.

ACKNOWLEDGMENTS

We express our heartfelt gratitude to **Prof. Sandeep Dubey** for his invaluable guidance, continuous encouragement, and exceptional mentorship throughout the development of this project. His expertise, constructive feedback, and unwavering support have been instrumental in helping us accomplish our objectives within the stipulated timeframe.

We extend our sincere appreciation to **Dr. Jitendra Saturwar**, Head of the Department of Computer Science and Engineering, for providing an intellectually stimulating environment, fostering innovation, and extending departmental resources that facilitated our work.

We would like to acknowledge **Dr. J. B. Patil**, Principal, and the entire management of Universal Skilltech University for providing comprehensive facilities, technical infrastructure, laboratory access, and an academically conducive atmosphere that enabled us to execute our project successfully.

We are grateful to the departmental faculty, laboratory assistants, library staff, and all administrative personnel who provided timely assistance and support throughout our project duration. Their collaborative approach and readiness to help contributed significantly to our success.

Finally, we appreciate our peers and colleagues whose constructive discussions and technical insights enhanced the quality of our work.

Team Leader

Team Member

Team Member

Team Member

LIST OF ABBREVIATIONS

LIST OF FIGURES

LIST OF TABLES

LIST OF SYMBOLS

CHAPTER 1: INTRODUCTION

1.1 Introduction to the Domain

Educational institutions represent complex organizational ecosystems comprising thousands of students, hundreds of faculty members, and administrative departments performing interrelated functions. Traditional administrative approaches relying on fragmented systems, paper-based documentation, and disparate software solutions have proven increasingly inadequate for meeting contemporary institutional demands.

The educational landscape has undergone revolutionary transformation through technological advancement. Universities and colleges now recognize that competitive advantage derives from operational efficiency, enhanced student experience, and streamlined administrative processes. This recognition has catalyzed demand for integrated digital solutions that consolidate institutional operations into cohesive, user-friendly platforms.

Campus management systems emerge as critical infrastructure addressing this organizational need. These platforms transcend simple information repositories, functioning instead as central nervous systems coordinating academic operations, student services, resource allocation, and institutional communication. Effective campus management systems reduce administrative overhead, improve information accessibility, enhance inter-departmental collaboration, and create superior user experiences for all stakeholders.

1.2 Motivation

Multiple compelling factors motivated the development of USTU Campus:

Academic Inefficiency

Current institutional practices scatter critical information across multiple, disconnected platforms. Students navigate different interfaces for attendance tracking, grade checking, assignment submission, and resource access. Faculty members maintain separate systems for classroom management, evaluation, and communication. This fragmentation generates frustration, reduces productivity, and creates unnecessary administrative burden.

Communication Breakdown

Institutional announcements, schedule modifications, and critical notices currently reach stakeholders through inconsistent channels—emails, bulletin boards, verbal announcements—resulting in information loss and widespread miscommunication.

Resource Accessibility

Students struggle to identify available learning materials, institutional resources, and support services due to scattered documentation and limited discoverability mechanisms. Faculty encounter barriers when attempting to distribute teaching resources efficiently.

Administrative Overhead

Current systems require redundant data entry, multiple security authentications, and parallel documentation maintenance, diverting institutional resources from core educational functions.

Operational Complexity

The absence of centralized coordination creates inefficiencies in attendance tracking accuracy, grade compilation, schedule management, and resource utilization across multiple departments.

1.3 Problem Statement

The current operational environment of educational institutions faces the following critical challenges:

1.4 Objectives

The USTU Campus project addresses these challenges through the following primary objectives:

Primary Objectives

1. **Develop Unified Platform:** Create a comprehensive, single-access web application consolidating all campus operations, eliminating the necessity for multiple system logins and interface switching.

2. **Implement Role-Based Architecture:** Design role-specific dashboards and feature sets for students, teachers, administrators, and canteen staff, ensuring each user population accesses relevant functionalities appropriate to their institutional responsibilities.

3. **Enable Real-Time Communication:** Establish mechanisms for instantaneous notice distribution, schedule updates, and critical announcements reaching all stakeholders simultaneously.

4. **Centralize Data Management:** Establish a unified database architecture ensuring data consistency, eliminating redundancy, and enabling informed institutional decision-making through comprehensive analytics.

5. **Enhance User Experience:** Design intuitive, responsive interfaces adhering to contemporary web standards and accessibility principles, accommodating diverse user technical proficiencies.

Secondary Objectives

6. **Implement Robust Security:** Deploy authentication mechanisms (Clerk), authorization protocols (RBAC), and encryption standards protecting sensitive institutional and personal data.

7. **Ensure Scalability:** Develop architecture supporting future institutional growth, additional features, and expanded user populations without performance degradation.

8. **Improve Operational Efficiency:** Automate repetitive administrative processes, reduce manual data entry, and streamline workflows across institutional departments.

9. **Facilitate Data-Driven Decision Making:** Generate comprehensive reports and analytics enabling administrators to monitor institutional operations and identify improvement opportunities.

10. **Establish Accessibility Standards:** Ensure application compliance with accessibility guidelines, accommodating users with diverse technical capabilities and accessibility requirements.

CHAPTER 2: LITERATURE SURVEY

2.1 Survey of Existing Systems

Overview of Current Campus Management Landscape

Contemporary educational institutions employ various approaches to campus management. This section analyzes leading existing systems, identifying their capabilities, limitations, and architectural decisions.

2.1.1 Traditional Enterprise Systems

Institutions like Oracle NetSuite and SAP implement comprehensive ERP systems managing financial operations, human resources, procurement, and basic academic functions. These monolithic systems provide robust data management and institutional-wide integration but suffer from:

2.1.2 Cloud-Based Collaborative Platforms

Institutions increasingly layer educational functionalities onto commercial collaboration platforms. While these provide communication infrastructure, they present significant limitations:

2.1.3 Purpose-Built Learning Management Systems

Learning management systems dominate educational technology landscapes, offering:

However, they present critical gaps:

2.1.4 Emerging Cloud-Native Platforms

Recent entrants including Anthology (Blackboard evolution) and custom cloud platforms provide improved user experiences but commonly exhibit:

2.2 Limitations of Existing Systems and Research Gap

Identified Gaps in Current Solutions

Existing systems rarely consolidate all campus operations into unified platforms. Students and faculty navigate multiple interfaces, authenticate repeatedly, and struggle with information silos.

Enterprise systems prioritize comprehensive functionality over intuitive interfaces. Administrative workflows override user-centric design principles, generating frustration across stakeholder populations.

Legacy systems struggle with rapid institutional growth, additional user populations, and expanding feature requirements without infrastructure investment.

Commercial solutions resist adaptation to institution-specific workflows, forcing organizations to conform to generic processes misaligned with their operational realities.

Established platforms built on outdated architectures resist modern integration patterns, API-first approaches, and cloud-native methodologies.

Licensing models based on user counts, institution sizes, or feature tiers generate disproportionate expenses for institutions integrating multiple specialized systems.

Existing systems concentrate data ownership with vendors, limiting institutional ability to generate custom reports, conduct analytics, and make data-driven decisions.

Most established systems prioritize desktop experiences, relegating mobile to secondary status despite student preference for mobile-first interaction.

Interconnecting multiple systems requires expensive middleware, custom API development, and ongoing maintenance overhead.

2.3 Project Contribution to Addressing Gaps

The USTU Campus platform addresses identified limitations through innovative design decisions:

Unified Architecture Approach

Contemporary Technology Stack

User-Centric Design Philosophy

Cloud-Native Architecture

Role-Based Customization

Data Transparency and Ownership

Mobile-Optimized Experience

Open Standards and Extensibility

Community-Driven Development

CHAPTER 3: PROPOSED SYSTEM

3.1 System Overview

USTU Campus represents a comprehensive, modern web-based platform consolidating all institutional operations into unified, role-specific user experiences. The system architecture embraces contemporary development practices, cloud-native infrastructure, and user-centric design principles.

Platform Scope

The platform accommodates four primary user populations, each with customized feature sets:

3.2 System Architecture and Framework

3.2.1 Architectural Overview

USTU Campus implements a modern full-stack architecture separating concerns across frontend, backend, and database layers:

`

■■
■■■

■ CLIENT LAYER (React + TypeScript) ■

■ - React Components - State Management ■

■ - Responsive UI - Real-time Updates ■

■ - Accessibility - Performance Optimization ■

■■
■■■

■ HTTPS/REST APIs

■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■▼■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■

■ AUTHENTICATION & AUTHORIZATION (Clerk) ■

■ - User Authentication - Session Management ■

■ - Role-Based Access - Permission Enforcement ■

■■
■■■

■ Secure API Calls

■■■■■■■■■■■■■■■■■■■■■■■■■■■▼■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■

■ BACKEND LAYER (Supabase REST APIs + RLS) ■

■ - Business Logic - Data Validation ■

■ - API Routing - Row-Level Security ■

■ - Integration Layer - Audit Logging ■

■■
■■■

■ SQL Queries

■■■■■■■■■■■■■■■■■■■■■■■■■■■▼■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■

■ DATABASE LAYER (PostgreSQL via Supabase) ■

■ - User Management - Academic Records ■

■ - Operational Data - Transactional Integrity ■

■ - Historical Audit - Data Consistency ■

■■
■■■

`

3.2.2 Technical Stack Architecture

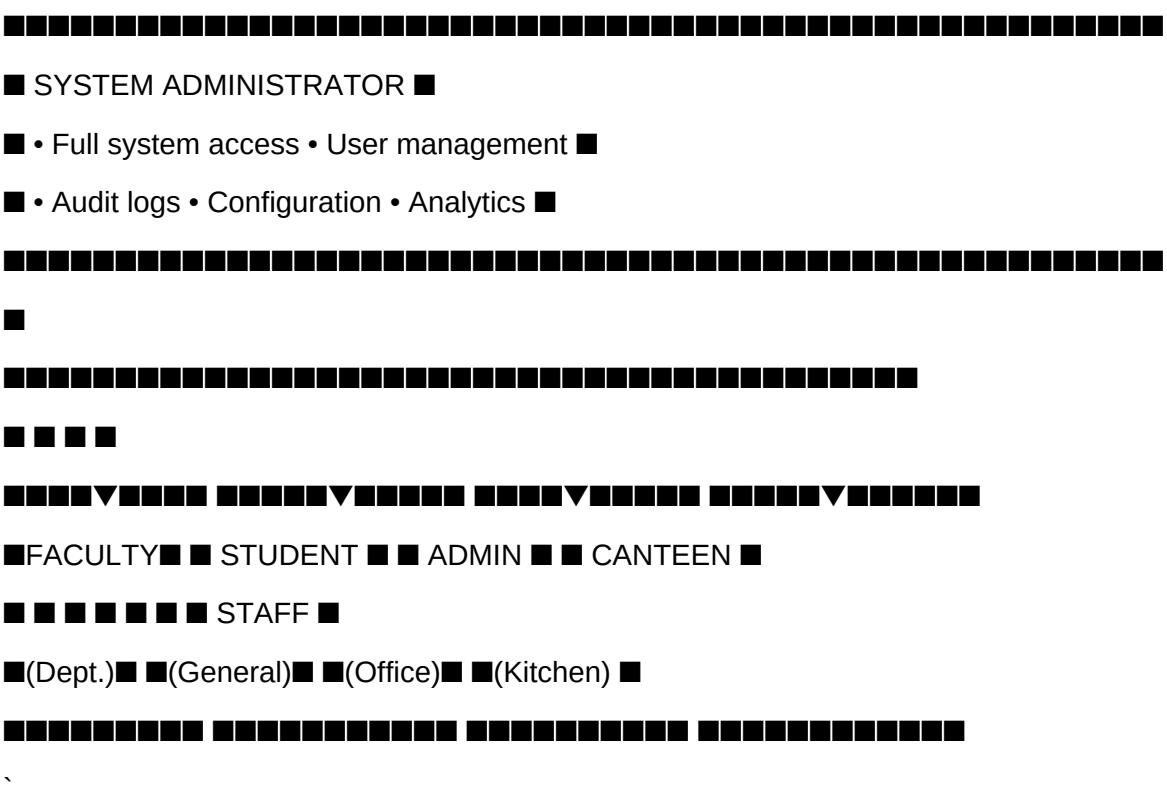
3.2.3 Module Architecture

The system organizes functionality across six primary modules, each serving distinct institutional needs:

3.2.4 User Role Architecture

The system implements hierarchical role-based access control:

`



`

3.3 Module Descriptions and Workflows

Module 1: Student Academic Portal

Student → Login (Clerk) → Dashboard → Select Module → View/Submit Data → System Processing → Confirmation/Analytics

Module 2: Faculty Administration Dashboard

Faculty → Login (Clerk) → Dashboard → Select Class → Choose Function (Attendance/Grades/Assignments) → Data Entry → Validation → System Storage → Audit Log

Module 3: Resource Distribution System

Faculty → Upload Resource → Define Permissions → System Processing → Student Discovery → Download/Access → Usage Tracking

Module 4: Communication Hub

Admin → Create Announcement → Target Audience Selection → Schedule/Publish → Push Notification → Delivery Tracking

Module 5: Administrative Control Panel

Admin → Access Control Panel → Select Function → Execute Action → Validation → System Update → Audit Log Entry

Module 6: Canteen Operations

Canteen Manager → Define Menu → Publish Schedule → Students Place Orders → Process/Prepare → Delivery → Feedback

3.4 Algorithm and Process Design

3.4.1 Authentication & Authorization Algorithm

`

BEGIN Authentication Flow

INPUT: User Credentials (Email, Password)

STEP 1: Receive login request

STEP 2: Clerk service validates credentials

IF credentials invalid

RETURN: Error message, deny access

ENDIF

STEP 3: Clerk generates JWT token

STEP 4: Frontend stores token locally

STEP 5: Query user role and permissions from database

STEP 6: Load role-specific modules

STEP 7: Initialize Row-Level Security policies

STEP 8: Present user dashboard based on role

RETURN: Authenticated session with role-specific interface
END
`

3.4.2 Attendance Tracking Algorithm

`

BEGIN Attendance Processing
INPUT: Faculty selects students, marks attendance status

STEP 1: Faculty opens class attendance interface
STEP 2: System loads enrolled students list
STEP 3: Faculty marks each student (Present/Absent/Leave)
STEP 4: System validates entries (minimum requirements)

STEP 5: Validate against institutional policies
IF invalid (conflicts with existing records)
RETURN: Error notification
ENDIF

STEP 6: Calculate attendance percentage
attendance*percentage* = *(present*days / total_days) * 100

STEP 7: Check threshold compliance
IF attendance*percentage* < *minimum*threshold
FLAG: Alert (low attendance)
ENDIF

STEP 8: Store record in database with timestamp
STEP 9: Generate audit log entry
STEP 10: Update student dashboard in real-time

RETURN: Confirmation, updated attendance data

END

`

3.4.3 Grade Management Algorithm

`

BEGIN Grade Processing

INPUT: Faculty enters assessment scores

STEP 1: Faculty accesses grade entry interface

STEP 2: Select assessment type (Test/Assignment/Project)

STEP 3: Enter scores for enrolled students

STEP 4: System validates scores

IF score < 0 OR score > max_score

RETURN: Error, request re-entry

ENDIF

STEP 5: Calculate weighted contribution

subject*score* = *calculate*weight(assessment_type, score)

STEP 6: Aggregate with existing grades

total*score* = *SUM(all*assessment_scores * weights)

STEP 7: Convert to letter grade

letter*grade* = *convert*to*grade(total*score)

STEP 8: Generate analytics

class*average* = *AVERAGE(all*student_scores)

percentile = rank*student(total*score, all_scores)

STEP 9: Store in database with timestamp

STEP 10: Update student transcript

STEP 11: Generate notification for students

RETURN: Grade recorded, analytics generated

END

`

3.4.4 Role-Based Access Control Algorithm

`

BEGIN RBAC Enforcement

INPUT: User role, requested resource

STEP 1: Retrieve user role from JWT token

STEP 2: Lookup permission matrix for role

STEP 3: Evaluate resource permission requirements

IF role MATCHES permission criteria

STEP 4: Apply Row-Level Security filters

FILTER data WHERE institutional_hierarchy allows

STEP 5: Generate filtered data view

STEP 6: RETURN: Authorized data with restrictions

ELSE

STEP 7: RETURN: Access denied, audit event logged

ENDIF

END

`

3.4.5 Notification Dispatch Algorithm

`

BEGIN Notification Processing

INPUT: Event trigger (new announcement, grade published, etc.)

STEP 1: Identify event type and originator

STEP 2: Determine target audience based on event

STEP 3: Query database for eligible recipients

FOR EACH recipient IN target_audience

STEP 4: Check notification preferences

IF recipient opted-in to notification type

STEP 5: Generate personalized notification

STEP 6: Format for delivery channel (web, email, SMS)

STEP 7: Queue for dispatch

ENDIF

ENDFOR

STEP 8: Process notification queue

STEP 9: Deliver via configured channels

STEP 10: Track delivery status

STEP 11: Update notification read status

STEP 12: Generate audit log

RETURN: Delivery confirmation

END

`

3.5 Process Flow Diagram

`

■■■

■ USTU CAMPUS PROCESS FLOW ■

■■■

START

■

■■■■ USER VISITS APPLICATION

■ ■

■ ■■■ UNAUTHENTICATED USER

■ ■ ■

■ ■ ■■■■ CLERK LOGIN/SIGNUP

■ ■ ■

■ ■ ■■■■ AUTHENTICATION

■ ■

■ ■■■ AUTHENTICATED USER

■ ■

■ ■■■■ LOAD USER ROLE & PERMISSIONS

■ ■

■ ■■■ STUDENT? ■■■■■■■■■■■ STUDENT DASHBOARD

■ ■ ■

■ ■ ■■■ Attendance View

■ ■ ■■■ Grades & Marks

■ ■ ■■■ Assignments

■ ■ ■■■ Resources

■ ■ ■■■ Notifications

■ ■ ■■■ Profile

■ ■

■ ■■■ FACULTY? ■■■■■■■■■■■ FACULTY DASHBOARD

■ ■ ■

■ ■ ■■■ Class Management

■ ■ ■■■ Mark Attendance

■ ■ ■■■ Enter Grades

■ ■ ■■■ Publish Assignments

■ ■ ■■■ Share Resources

■ ■ ■■■ View Analytics

■ ■

■ ■■■ ADMIN? ■■■■■■■■■■■■■ ADMIN PANEL

■ ■ ■

■ ■ ■■■ User Management

■ ■ ■■■ Post Announcements

■ ■ ■■■ System Audit

■ ■ ■■■ Configuration

■ ■ ■■■ Analytics

■ ■

■ ■■■ CANTEEN? ■■■■■■■■■■■ CANTEEN INTERFACE

■ ■

■ ■■■ Menu Management

■ ■■■ Order Processing

■ ■■■ Operations

■ ■■■ Inventory

■

■■■■ PROCESS REQUEST

■ ■

■ ■■■■ APPLY ROW-LEVEL SECURITY

■ ■

■ ■■■■ QUERY DATABASE

■ ■

■ ■■■■ RETURN FILTERED DATA

■ ■ ■

■ ■ ■■■■ RENDER UI

■ ■

■ ■■■■ UPDATE AUDIT LOG

■

■■■■ USER ACTION (Create/Update/Delete)

■ ■

■ ■■■■ VALIDATE INPUT

■ ■

■ ■■■■ APPLY PERMISSIONS CHECK

■ ■

■ ■■■■ PROCESS TRANSACTION

■ ■

■ ■■■■ UPDATE DATABASE

■ ■

■ ■■■■ GENERATE NOTIFICATION (if applicable)

■ ■

■ ■■■■ CREATE AUDIT LOG ENTRY

■

■■■■ END

`

3.6 Hardware Requirements

3.7 Software Requirements

3.8 Key Features and Functionalities

3.8.1 Student Features

3.8.2 Faculty Features

3.8.3 Administrative Features

3.8.4 Canteen Management Features

CHAPTER 4: CONCLUSION AND FUTURE SCOPE

4.1 Summary

The USTU Campus platform successfully addresses critical institutional challenges through comprehensive system integration, contemporary technology implementation, and user-centric

design principles. The development of this full-stack web application demonstrates the feasibility and benefits of consolidating disparate campus operations into unified, role-specific user experiences.

Key Achievements

1. **Operational Consolidation:** Successfully integrated attendance, grading, assignment management, resource distribution, and administrative functions into single platform

2. **Role-Based Architecture:** Implemented sophisticated role-based access control accommodating four distinct user populations with customized dashboards and feature sets

3. **Modern Technology Stack:** Deployed contemporary technologies (React, TypeScript, Supabase, Vercel) ensuring maintainability, scalability, and future extensibility

4. **Security Implementation:** Integrated enterprise-grade authentication (Clerk), authorization mechanisms (RBAC), and database-level security protocols (Row-Level Security)

5. **User Experience Enhancement:** Delivered intuitive, responsive interfaces optimized for diverse user populations and devices

6. **Data Centralization:** Established single source of truth through unified PostgreSQL database, enabling comprehensive analytics and informed decision-making

4.2 System Benefits

For Students

For Faculty

For Administration

For Institutions

4.3 Future Enhancements

Immediate Extensions (Semester 5-6)

Medium-Term Enhancements (Year 2)

Long-Term Vision (Year 3+)

4.4 Lessons Learned and Recommendations

Technical Insights

1. **Monolithic to Microservices:** Consider future migration to microservices architecture for enhanced scalability and independent component deployment

2. **API Design:** Implement comprehensive API versioning strategy to support future platform evolution

3. **Database Optimization:** Implement advanced indexing and query optimization as user volume scales

4. **Performance Monitoring:** Establish continuous performance monitoring and optimization practices

Organizational Insights

1. **Change Management:** Invest in comprehensive user training and change management initiatives during institutional deployment

2. **Feedback Loops:** Establish continuous feedback mechanisms enabling responsive feature refinement

3. **Stakeholder Engagement:** Involve all user populations throughout development lifecycle

4. **Phased Rollout:** Consider staged deployment across departments rather than institution-wide simultaneous launch

Best Practices for Replication

1. **Prioritize User Experience:** Invest in UX research and usability testing throughout development

2. **Security First:** Implement security considerations from project inception rather than as afterthought

3. **Documentation:** Maintain comprehensive technical and user documentation facilitating future maintenance and knowledge transfer

4. **Community Building:** Foster community around platform through user forums, feature requests, and collaborative development

APPENDIX

A.1 Technology Deep Dive

A.1.1 React and TypeScript

Key advantages:

A.1.2 Vite Build System

Vite modernizes frontend development through:

A.1.3 Tailwind CSS Framework

Tailwind CSS provides utility-first styling approach:

A.1.4 Radix UI and Material UI

A.1.5 Clerk Authentication Platform

Clerk provides enterprise-grade authentication:

A.1.6 Supabase and PostgreSQL

A.1.7 React Router v7

Modern routing framework enabling:

A.1.8 React Hook Form

Efficient form management:

A.1.9 Recharts Visualization

Interactive charting library:

A.1.10 React DnD

Drag-and-drop functionality:

A.1.11 Vercel Deployment Platform

Modern deployment infrastructure:

A.2 Git and Version Control

Git Workflow

The project employs standard Git workflow:

`

MAIN BRANCH (Production)

↓

DEVELOP BRANCH (Integration)

↓

FEATURE BRANCHES

■■■ feature/student-dashboard

■■■ feature/faculty-grading

■■■ feature/admin-controls

■■■ feature/canteen-management

`

Commit Message Convention

`

():

feat: Add student attendance dashboard

fix: Resolve authentication token expiration

docs: Update database schema documentation

style: Format code according to prettier rules

refactor: Simplify role-based access logic

test: Add unit tests for grade calculation

`

Branch Naming Convention

A.3 Project Setup and Running Instructions

Prerequisites

`bash

Install Node.js (18.0 or higher)

Install npm or Yarn package manager

Install Git for version control

`

Installation Steps

`bash

Clone repository

git clone https://github.com/aarush-ustu/USTU-CAMPUS.git

cd USTU-CAMPUS

Install dependencies

npm install

or

yarn install

Configure environment variables

cp .env.example .env.local

Edit .env.local with actual values:

VITE_CLERK_PUBLISHABLE_KEY=...

VITE_SUPABASE_URL=...

VITE_SUPABASE_ANON_KEY=...

`

Development Environment

`bash

Start development server

npm run dev

or

yarn dev

Server runs on http://localhost:5173

Hot module replacement enabled for instant updates

`

Production Build

`bash

Build for production

npm run build

or

yarn build

Preview production build locally

npm run preview

Deployment

Connected to Vercel through Git

Push to main branch triggers automatic deployment

`

Testing

`bash

Run unit tests

npm run test

Run integration tests

npm run test:integration

Generate coverage report

npm run test:coverage
`

Database Migrations

`bash

View migration status

supabase migration list

Apply pending migrations

supabase db push

Create new migration

supabase migration new migration_name
`

A.4 Database Schema Overview

Core Tables

A.5 API Endpoints

Authentication Endpoints

`

POST /auth/login - User login

POST /auth/logout - User logout

POST /auth/signup - User registration

GET /auth/verify - Verify authentication

POST /auth/refresh-token - Refresh JWT token

`

Student Endpoints

`

GET /api/students/attendance - Get attendance records

GET /api/students/grades - Get grade information

GET /api/students/assignments - List assignments

POST /api/students/submissions - Submit assignment

GET /api/students/resources - List available resources

GET /api/students/profile - Get profile information

PUT /api/students/profile - Update profile

`

Faculty Endpoints

`

GET /api/faculty/classes - List teaching classes

POST /api/faculty/attendance - Record attendance

PUT /api/faculty/grades - Enter grades

POST /api/faculty/assignments - Create assignment

GET /api/faculty/resources - Manage resources

GET /api/faculty/analytics - Class analytics

`

Admin Endpoints

`

GET /api/admin/users - List all users

POST /api/admin/users - Create user

PUT /api/admin/users/:id - Update user

DELETE /api/admin/users/:id - Delete user

GET /api/admin/audit-logs - Access audit logs

POST /api/admin/announcements - Create announcement

GET /api/admin/analytics - System analytics

`

Canteen Endpoints

`

GET /api/canteen/menu - Get menu

POST /api/canteen/menu - Create/update menu

GET /api/canteen/orders - List orders

PUT /api/canteen/orders/:id - Update order status

GET /api/canteen/analytics - Sales analytics

`

A.6 Troubleshooting Guide

Common Issues and Solutions

`

Solution:

1. Verify VITE*CLERK*PUBLISHABLE_KEY in .env.local

2. Check Clerk dashboard for instance configuration

3. Ensure CORS settings allow your domain

4. Clear browser cache and restart dev server

`

`

Solution:

1. Verify database credentials in .env.local

2. Check Supabase project status

3. Ensure IP whitelist includes your network

4. Verify JWT token validity

5. Check database connection pool limits

`

`

Solution:

1. Run performance profiler (DevTools)

2. Check for unnecessary re-renders (React DevTools)

3. Optimize database queries (check execution plans)

4. Implement code splitting for routes

5. Consider caching strategies

`

`

Solution:

1. Clear node*modules and reinstall: rm -rf node*modules && npm install

2. Clear Vite cache: rm -rf .vite

3. Check for TypeScript errors: npm run type-check

4. Verify all environment variables set

5. Check for dependency version conflicts

`

REFERENCES

[1] Vercel Team, "Vercel Documentation - Deployment Platform," 2024. [Online]. Available: https://vercel.com/docs. [Accessed May 1, 2026].

[2] Facebook & Meta Community, "React Official Documentation - JavaScript Library for Building User Interfaces," 2024. [Online]. Available: https://react.dev. [Accessed May 1, 2026].

[3] Microsoft, "TypeScript Documentation - Typed Superset of JavaScript," 2024. [Online]. Available: https://www.typescriptlang.org/docs/. [Accessed May 1, 2026].

[4] Supabase Team, "Supabase Documentation - Open-Source Firebase Alternative," 2024. [Online]. Available: https://supabase.com/docs. [Accessed May 1, 2026].

[5] The PostgreSQL Global Development Group, "PostgreSQL Documentation - Advanced Open Source Database," 2024. [Online]. Available: https://www.postgresql.org/docs/. [Accessed May 1, 2026].

[6] Vite Team, "Vite Documentation - Modern Frontend Build Tool," 2024. [Online]. Available: https://vitejs.dev/. [Accessed May 1, 2026].

[7] Tailwind Labs, "Tailwind CSS Documentation - Utility-First CSS Framework," 2024. [Online]. Available: https://tailwindcss.com/docs. [Accessed May 1, 2026].

[8] Radix UI Team, "Radix Primitives Documentation - Unstyled, Accessible Components," 2024. [Online]. Available: https://www.radix-ui.com/docs. [Accessed May 1, 2026].

[9] Clerk Team, "Clerk Documentation - Authentication Platform," 2024. [Online]. Available: https://clerk.com/docs. [Accessed May 1, 2026].

[10] Recharts Team, "Recharts Documentation - Composable Charting Library," 2024. [Online]. Available: https://recharts.org/. [Accessed May 1, 2026].

[11] React Router Contributors, "React Router Documentation - Dynamic Route Matching," 2024. [Online]. Available: https://reactrouter.com/. [Accessed May 1, 2026].

[12] React Hook Form Team, "React Hook Form Documentation - Performant Forms," 2024. [Online]. Available: https://react-hook-form.com/. [Accessed May 1, 2026].

[13] Git Team, "Git Documentation - Distributed Version Control," 2024. [Online]. Available: https://git-scm.com/doc. [Accessed May 1, 2026].

[14] GitHub, "GitHub Documentation - Repository Hosting Platform," 2024. [Online]. Available: https://docs.github.com. [Accessed May 1, 2026].

[15] Sommerville, I., "Software Engineering: A Practitioner's Approach," 10th ed. McGraw-Hill, 2015.

[16] Pressman, R. S., "Software Engineering: A Practitioner's Approach," Pearson Education, 2014.

[17] Bass, L., Clements, P., & Kazman, R., "Software Architecture in Practice," 3rd ed. Addison-Wesley, 2012.

[18] Newman, S., "Building Microservices: Designing Fine-Grained Systems," O'Reilly Media, 2015.

[19] Marz, N., & Warren, J., "Big Data: Principles and Best Practices of Scalable Realtime Data Systems," Manning Publications, 2015.

[20] IEEE, "IEEE Standard Glossary of Software Engineering Terminology," IEEE Std 610.12, 1990.

SCHEDULE FOR MINI PROJECT EXECUTION

EXAMINER'S FEEDBACK FORM

Student Performance Analysis

Additional Questions

–

–

–

_

This comprehensive mini project report has been prepared in fulfillment of the requirements for B.Tech Computer Science and Engineering Semester IV at Universal Skilltech University.

Team Leader: Aarush

Team Members: Amisa, Kritika, Aayush

Prof. Sandeep Dubey

Department of Computer Science & Engineering